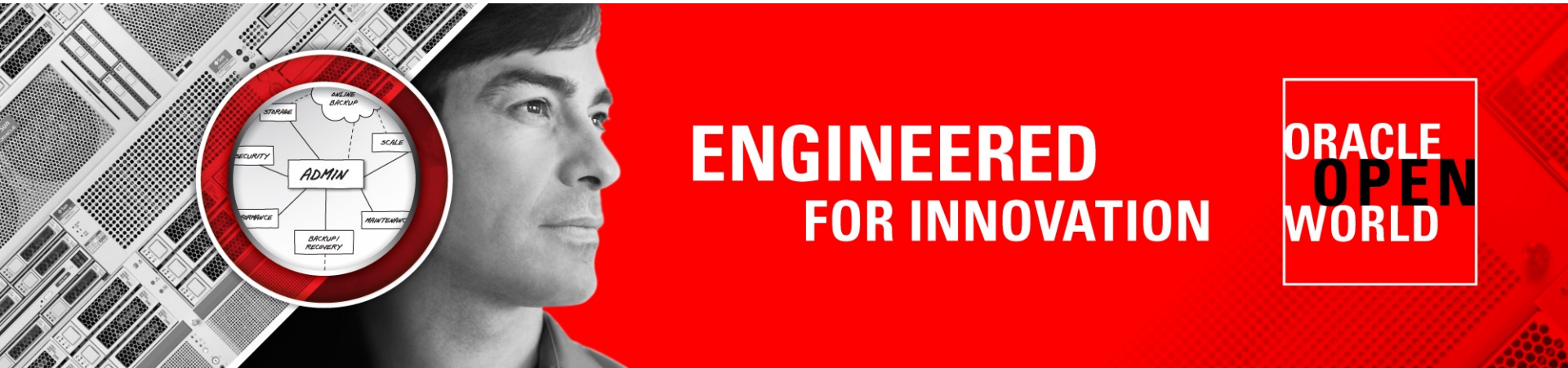




Cours PL/SQL Partie 3

Présenté par: Mme Olfa DRIDI

Contenu

- Les procédures
- Les fonctions
- Les structures conditionnelles
- Les structures itératives
- Les curseurs
- Les exceptions
- Les déclencheurs

Procédures stockées

- Une procédure est un bloc nommé, éventuellement paramétré, qu'on peut exécuter à la demande.
- Une procédure est un code PL/SQL défini par l'utilisateur et stocké dans la base de données. Ce qui permet d'éliminer la redondance de code.

Avantages:

- Le code SQL est précompilé.
- Exécution plus rapide (puisque stockée dans le serveur)
- Moins de code redondant

Avantages des procédures stockées

- Vitesse : programme compilé et optimisé
 - Une requête SQL normale est interprétée et optimisée à chaque exécution
- Intégrité : encapsulation des données
 - Droit d'exécution et plus de manipulation
 - Les règles de gestion sont données sur le serveur en un seul exemplaire
- Performance : moins de transfert réseau
 - Plus de transfert de bloc de programme
 - Une procédure pour plusieurs utilisateurs
- Abstraction : augmentation du niveau d'abstraction des développeurs Client
- Performance :
 - Extensibilité, Modularité, Réutilisation, Maintenance

Déclaration d'une procédure stockée

```
CREATE [OR REPLACE] PROCEDURE <nom_procedure>
[(variable1 [IN|OUT|IN OUT ]type1, ..., variablen typen
[IN |OUT|IN OUT]))] IS | AS
...
-- déclarations des variables et
-- curseurs utilisées dans le corps de la procédure
BEGIN
....
-- instructions SQL ou PL/SQL
EXCEPTION
....
END;
/
```

Exemple 1 de procédure stockée

inscription d'un étudiant

```
CREATE PROCEDURE inscription (pnom IN
etudiant.nom%TYPE,
... ,pdip diplome.idDip%TYPE)
AS
CURSOR uv_ins IS SELECT c.iduv AS uv FROM composition c
WHERE c.idDip=pdip;
BEGIN
DBMS_OUTPUT.PUT_LINE('Début inscription de '||pnom);
INSERT INTO etudiant VALUES(seqEtu.NEXTVAL,pnom,...,pdip);
FOR uv_1 IN uv_ins LOOP
INSERT INTO inscrire VALUES(seqEtu.CURRVAL,uv_1.uv);
END LOOP;
DBMS_OUTPUT.PUT_LINE('Transaction réussie');
COMMIT;
EXCEPTION
....
END;
/
```

Exemple 1 : appel de la procédure

- A partir de sqlplus

```
ACCEPT vnom PROMPT 'Entrer le nom : '  
.....  
EXECUTE inscription('&vnom',....., '&vdip');
```

- A partir de PL/SQL

```
inscription(nom,....., dip);
```

Les Fonctions stockées

- Comme une procédure mais qui ne retourne qu'un seul résultat
- Même structure d'ensemble qu'une procédure
- Utilisation du mot clé **RETURN** pour retourner le résultat
- Appel possible à partir de :
 - Une requête SQL normale
 - Un programme PL/SQL
 - Une procédure stockée ou une autre fonction stockée
 - Un programme externe comme pro*c

Déclaration d'une fonction stockée

```
CREATE [OR REPLACE] FUNCTION nom_fonction
[(paramètre1 [IN|OUT|IN OUT] type1, ..... , paramètren
  typen)]
RETURN type_résultat IS | AS
-- déclarations de variables, curseurs et exceptions
BEGIN
-- instructions PL et SQL

RETURN(variable);
END;
/
```

Exemple 1 de fonction stockée

```
CREATE OR REPLACE FUNCTION moy_points_marques
(eqj joueur.ideq%TYPE)
RETURN NUMBER IS
moyenne_points_marques NUMBER(4,2);
BEGIN
SELECT AVG(totalpoints) INTO moyenne_points_marques
FROM joueur WHERE ideq=eqj;
RETURN(moyenne_points_marques);
END;
/
```

Utilisation d'une fonction

- A partir d'une requête SQL

```
SELECT moy_points_marques('e1') FROM dual;
```

```
SELECT nomjoueur FROM joueur WHERE  
totalpoints > moy_points_marques('e1');
```

- A partir d'une procédure ou fonction

```
BEGIN  
.....  
IF moy_points_marques(equipe) > 20 THEN .....  
END;
```

La table DUAL

- Elle est créée par défaut par Oracle et accessible par tous les utilisateurs.
- Elle ne contient qu'un champ DUMMY avec la valeur 'C'
- Elle est utilisée pour obtenir des valeurs calculées sur la base des constantes
- *Exemple :*

```
SELECT SYSDATE FROM DUAL;
```

Exemple 2 de fonction stockée

```
CREATE OR REPLACE FUNCTION bon_client
(pidclient NUMBER, pchiffre NUMBER)
RETURN BOOLEAN IS
total_chiffre NUMBER;
BEGIN
SELECT SUM(qte*prix_unit) INTO total_chiffre
FROM commande WHERE idclient=pidclient;
IF total_chiffre > pchiffre THEN
    RETURN(TRUE);
ELSE RETURN(FALSE);
END IF;
END;
```



```
BEGIN
.....
IF bon_client(client,10000) THEN .....
.....
```

Les Paquetages

- Ensemble de programmes ayant un lien logique entre eux
- Exemple : package étudiant qui peut regrouper tous les programmes écrits sur les étudiants
- Début de l'approche objet avec les méthodes associées à une classe (MEMBER en Objet-Relationnel)

Structure d'un paquetage

- Partie 'visible' ou spécification
 - Interface accessible au programme appelant
 - Ne contient que les déclarations des procédures ou fonctions publiques
 - Variables globales et session
 - Curseurs globaux
- Partie 'cachée' ou body
 - Corps des procédures ou des fonctions citées dans la partie spécification
 - Nouvelles procédures ou fonctions privées accessibles uniquement par des procédures ou fonctions du paquetage

Déclaration d'un paquetage

partie spécification

```
-- Partie Spécification

CREATE [OR REPLACE] PACKAGE nom_package AS
  Procedure Procédure1(liste des paramètres);
  .....
  Function Fonction1(liste des paramètres);
  .....
  Variable_globale1 type1;
  .....
  CURSOR Curseur_global1 IS .....
  .....
END nom_package;
/
```


Déclaration d'un paquetage

partie body

```
-- Partie body
CREATE [OR REPLACE] PACKAGE BODY nom_package AS
Procedure Procédure1(liste des paramètres) IS
.....
BEGIN
.....
END Procédure1;
Function Fonction1(liste des paramètres)
    RETURN type IS
.....
BEGIN
.....
RETURN (.....) ;
END Fonction2;
END nom_package;
/
```

Exemple : package 'étudiant' (1)

```
CREATE PACKAGE etudiant AS
-- Procédure publique inscription
PROCEDURE inscription (pnom etudiant.nom%TYPE,
... ,pdip diplome.idDip%TYPE);
-- Procédure publique suppression
PROCEDURE suppression(pidetu NUMBER);
END nom_package;
/

CREATE PACKAGE BODY etudiant AS
PROCEDURE inscription (pnom etudiant.nom%TYPE,
... ,pdip diplome.idDip%TYPE) IS
CURSOR uv_ins IS SELECT c.iduv AS uv FROM
  composition c
WHERE c.idDip=pdip;
BEGIN
```

Exemple : package 'étudiant' (2)

```
INSERT INTO etudiant
  VALUES (seqEtu.NEXTVAL, pnom, ..., pdip);
FOR uv_1 IN uv_ins LOOP
  INSERT INTO inscrire VALUES (seqEtu.CURRVAL, uv_1.uv);
END LOOP;
DBMS_OUTPUT.PUT_LINE('Transaction réussie');
COMMIT;
EXCEPTION      ....
END inscription;
-- fonction privée inscrit_uv
FUNCTION inscrit_uv(pidetu NUMBER) RETURN BOOLEAN IS
  nbre_ins NUMBER;
BEGIN
  SELECT COUNT(*) INTO nbre_ins FROM inscrire WHERE
  Idetu=pidetu;
  IF nbre_ins>0 THEN RETURN(TRUE) ELSE RETURN(FALSE)
END IF;
END inscrit_uv;
```

Exemple : package 'étudiant' (3)

```
PROCEDURE suppression (pidetu NUMBER) AS
BEGIN
IF inscrit_uv(pidetu) THEN
DBMS_OUTPUT.PUT_LINE('Cet étudiant est
inscrit à des UV');
DBMS_OUTPUT.PUT_LINE('Impossible de le supprimer');
ELSE
DELETE FROM etudiant WHERE idetu=pidetu;
DBMS_OUTPUT.PUT_LINE('Etudiant supprimé');
COMMIT;
END IF;
END suppression;
END etudiant;
/
```

Appel d'un programme d'un package

- A partir des SQL

```
ACCEPT vnom PROMPT 'Entrer le nom : '  
.....  
EXECUTE etudiant.inscription('&vnom',....., '&vdip');
```

- A partir d'un autre package

```
etudiant.inscription(nom,....., dip);
```

- Uniquement les programmes PUBLICS



Les curseurs

Curseurs

- Toutes les requêtes SQL sont associées à un curseur
- Ce curseur représente la zone mémoire utilisée pour analyser et exécuter la requête
- Le curseur peut être implicite (pas déclaré par l'utilisateur) ou explicite
- Les curseurs explicites servent à retourner plusieurs lignes avec un select
- On distingue deux types de curseurs :
 - Les curseurs explicites
 - Les curseurs implicites

Curseurs explicites

- Un curseur est une zone mémoire de taille fixe, utilisée par le moteur SQL pour analyser et interpréter un ordre SQL
- Un curseur explicite, contrairement au curseur implicite est géré par l'utilisateur pour traiter un ordre Select qui ramène plusieurs lignes.
- Tout curseur explicite géré dans la section exécution doit avoir été déclaré dans la section déclarative
- C'est une commande SELECT pouvant ramener plusieurs lignes et qui est totalement à la charge du développeur.
- Un curseur explicite doit être explicitement :
 - Déclaré dans la section DECLARE
 - Géré par le développeur dans la section exécutable.
 - La gestion d'un curseur consiste à exécuter les opérations : ouverture du curseur, lecture et traitement des lignes, fermeture.

Démarche générale des curseurs

- Déclaration du curseur : DECLARE
 - Ordre SQL sans exécution
- Ouverture du curseur : OPEN
 - SQL 'monte' les lignes sélectionnées en SGA
 - Verrouillage préventif possible (voir + loin)
- Sélection d'une ligne : FETCH
 - Chaque FETCH ramène une ligne dans le programme client
 - Tant que ligne en SGA : FETCH
- Fermeture du curseur : CLOSE
 - Récupération de l'espace mémoire en SGA

Gestion 'classique' d'un curseur

```
DECLARE
CURSOR c1 IS SELECT nom,moyenne FROM etudiant ORDER BY 1;
vnom          etudiant.nom%TYPE;
vmoyenne      etudiant.moyenne%TYPE;
e1 NUMBER:= 0;
e2 NUMBER := 0;
BEGIN
    OPEN c1;
    FETCH c1 INTO vnom,vmoyenne;
    WHILE c1%FOUND LOOP
        IF vmoyenne < 10 THEN e1:=e1+1;
            INSERT INTO liste_refus VALUES (vnom);
        ELSE e2:=e2+1;
            INSERT INTO liste_recus VALUES (vnom);
        END IF;
        FETCH c1 INTO vnom,vmoyenne;
    END LOOP;
    CLOSE c1;
    DBMS_OUTPUT.PUT_LINE (TO_CHAR(e2)||'ayant réussi');
    DBMS_OUTPUT.PUT_LINE (TO_CHAR(e1)||'ajournés ');
    COMMIT;
END;
```

Les variables système des Curseurs

- Curseur%FOUND
 - Variable booléenne
 - Curseur toujours 'ouvert' (encore des lignes)
- Curseur%NOTFOUND
 - Opposé au précédent
 - Curseur 'fermé' (plus de lignes)
- Curseur%COUNT
 - Variable number
 - Nombre de lignes déjà retournées
- Curseur%ISOPEN
 - Booléen : curseur ouvert ?

Curseurs implicites

- Les curseurs implicites sont plus facile à utiliser que ceux explicites. On ne déclare pas de curseur, ni on l'ouvre ni on le ferme. On utilise toujours la boucle FOR.
- Elle *simplifie* la programmation car elle évite d'utiliser explicitement les instruction OPEN, FETCH et CLOSE.
- En plus elle déclare *implicitement* une variable de type ROW associée au curseur.

Gestion 'automatique' des curseurs

```
DECLARE
CURSOR c1 IS SELECT nom, moyenne FROM etudiant ORDER BY 1;
-- PAS DE DECLARATION DE VARIABLE DE RECEPTION
e1 ,e2 NUMBER :=0;
BEGIN
    --PAS D'OUVERTURE DE CURSEUR
    --PAS DE FETCH
    FOR c1_ligne IN c1 LOOP
        IF c1_ligne.moyenne < 10 THEN e1:=e1+1;
        INSERT INTO liste_refus VALUES(c1_ligne.nom);
        ELSE    e2:=e2+1;
        INSERT INTO liste_recus VALUES(c1_ligne.nom);
        END IF;
    END LOOP;
    --PAS DE CLOSE
    DBMS_OUTPUT.PUT_LINE(TO_CHAR(e2)||'réussis');
    DBMS_OUTPUT.PUT_LINE(TO_CHAR(e1)||'ajournés');
    COMMIT;
END;
```

Variable STRUCT de réception

Curseurs et Tableaux

exemple final

```
DECLARE
CURSOR c1 IS SELECT nom, moyenne FROM etudiant
WHERE moyenne>=10 ORDER BY 2 DESC;
TYPE type_nom_tab IS TABLE OF etudiant.nom%TYPE
INDEX BY BINARY_INTEGER;
tab_noms type_nom_tab;
i,j NUMBER;
BEGIN  /* Remplissage tableau */
        i:=1;
        FOR c1_ligne IN c1 LOOP
            tab_noms(i):= c1_ligne.nom;
            i:=i+1;
        END LOOP; /* Affichage du tableau */
        FOR j IN 1..i-1 LOOP
            DBMS_OUTPUT.PUT_LINE('Rang : '||TO_CHAR(j)||
            'Etudiant : '||tab_noms(j));
        END LOOP;
END;
```

```
DECLARE

BEGIN

FOR curseur_EMPLOYE IN (SELECT * FROM EMP WHERE JOB =
                        'MANAGER')

LOOP DBMS_OUTPUT.PUT_LINE('L'employé' || curseur_EMPLOYE.ENAME
|| ' a un salaire de ' || curseur_EMPLOYE.SAL || '. Le numéro
de son Manager est ' || curseur_EMPLOYE.MGR);

END LOOP;

END;
```

Curseur paramétré

- Un curseur paramétré peut servir plusieurs fois avec des valeurs des paramètres différentes.
- On doit fermer le curseur entre chaque utilisation de paramètres différents si on utilise pas la boucle FOR dédiée.

```
DECLARE
CURSOR c(p_dep EMP.dep%TYPE) IS SELECT dep , nom FROM emp WHERE
dep = p_dep;
BEGIN
FOR v_employe IN c(1) LOOP
dbms_output.put_line('Dep_1_:_' || v_employe.nom);
END LOOP;
FOR v_employe IN c(2) LOOP
dbms_output.put_line('Dep_2_:_' || v_employe.nom);
END LOOP;
END ;
```


Exercice

- Écrire un bloc qui permet d'afficher le nom et le salaire des managers ainsi que le numéro de leurs managers.

DECLARE

```
V_ENAME EMP.ENAME%TYPE;
```

```
V_MGR EMP.MGR%TYPE;
```

```
V_SAL EMP.SAL%TYPE;
```

```
CURSOR curseur_EMPLOYE IS SELECT ENAME, MGR, SAL FROM EMP WHERE  
JOB = 'MANAGER';
```

BEGIN

```
OPEN curseur_EMPLOYE ;
```

```
LOOP FETCH curseur_EMPLOYE INTO V_ENAME, V_MGR, V_SAL;
```

```
DBMS_OUTPUT.PUT_LINE('L'employé ' || V_ENAME || ' a un salaire  
de ' || V_SAL || '. Le numéro de son Manager est ' || V_MGR);
```

```
EXIT WHEN NOT curseur_EMPLOYE%FOUND (curseur_EMPLOYE%NOTFOUND);
```

```
END LOOP;
```

```
CLOSE curseur_EMPLOYE ;
```

```
END;
```


DECLARE**V_EMP EMP%ROWTYPE;**

```
CURSOR curseur_EMPLOYE IS SELECT * FROM EMP
                                WHERE JOB = 'MANAGER';
```

BEGIN

OPEN curseur_EMPLOYE ;

LOOP FETCH curseur_EMPLOYE INTO V_EMP ;

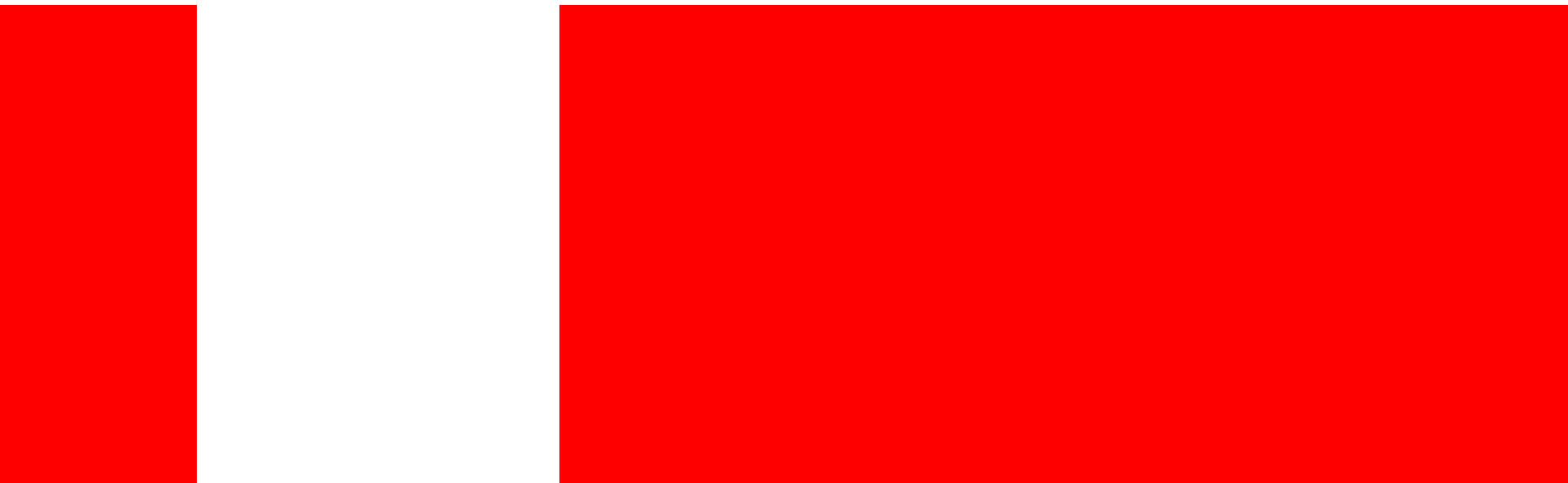
```
DBMS_OUTPUT.PUT_LINE('L''employé ' || V_EMP.ENAME || ' a
un salaire de ' || V_EMP.SAL || '. Le numéro de son
Manager est ' || V_EMP.MGR);
```

EXIT WHEN NOT curseur_EMPLOYE%FOUND;

END LOOP;

CLOSE curseur_EMPLOYE ;

END;



ORACLE®

Questions ???